

Herald



Herald

Field	Value
Revision	5
Created	2026-05-15
Last modified	2026-05-31
Status	active
Status summary	r5: added an "Operator setup guides" note + doc-link that subscribers speak plain natural language (no command syntax) and the System determines intent via the three-tier discipline (command-recognition fast-path → Claude Code intent inference → clarify reply-tag-and-ask fallback) — authoritative contract docs/design/INTENT_RECOGNITION.md, detail in docs/guides/MESSENGER_CHANNELS.md §6B. Prior r4: added a credentials-guide bullet + doc-link for the HERALD_<CHANNEL>_OPERATOR_USERNAME operator env var and the participant/attribution contract (docs/design/PARTICIPANT_ATTRIBUTION.md) driving created_by/assigned_to attribution + notification @-tagging. Prior r3: updated spec links to V4 (active) + archive/ for V1/V2; repo-layout block reflects current docs/specs/mvp/ tree.
Issues	none
Issues summary	—
Fixed	spec-path references updated to specification.V4.md path
Fixed summary	aligned README with the V1→V2→V3 supersession chain
Continuation	—

Table of contents

- [Status](#)
- [Mission](#)
- [Deployment model](#)
- [Governance — Helix Constitution inheritance](#)
- [Repository layout](#)
- [Quickstart for developers and agents](#)
 - [1. Place a copy of the Helix Constitution alongside Herald](#)
 - [2. Verify the inheritance contract](#)

- [3. Read the inherited rules](#)
- [Operator setup guides](#)
 - [Messengers](#)
 - [LLM / agent dispatchers](#)
 - [Per-flavor operator guides](#)
 - [Credentials master guide](#)
- [Mirror & push convention](#)
- [License](#)
- [Contact / contribution](#)

Ingesting system events and reliably fanning them out to multiple notification channels so every alert reaches the right destination without confusion.

Status

Herald is in **active multi-wave implementation** (as of 2026-05-31) — waves 2 through 7 have shipped and the latest tag is **v0.6.0** (2026-05-28). The repository contains:

- This README.
- The current specification at [docs/specs/mvp/specification.V4.md](#) — comprehensive: project-integration contract, inbound processing pipeline, LLM/agent dispatch with Claude Code, tri-stage reply protocol, versioned reports, multi-format outbound attachments, the refined flavors. V1–V3 preserved in [docs/specs/mvp/archive/](#) for traceability.
- **18 Go workspace modules** (10 shared/foundation + 8 flavor binaries `pherald/` `sherald/` `cherald/` `bherald/` `rherald/` `iherald/` `scherald/` `qaherald/`). Live and tested: the `pherald listen` inbound runtime, the Telegram + Slack channel adapters, the Claude Code dispatcher, `commons_storage` (pgx + River + Redis), and the Gin / v1 / * REST API.
- Landed cross-cutting mandates: participant-attribution (§11.4.104), natural-language intent-recognition (§11.4.105), and the Docs Chain documentation-sync integration (§11.4.106).
- Project-specific Constitution + operator/agent guides under [docs/guides/](#).
- Mirror declarations at [upstreams/](#) — one shell script per host that exports `UPSTREAMABLE_REPOSITORY`.
- The inheritance gate + paired mutation meta-test at [tests/](#), plus `scripts/e2e_bluff_hunt.sh` (the canonical anti-bluff end-to-end smoke).
- `.gitignore` tuned for Go (`*.test`, `go.work*`, `coverage.*`) and `.DS_Store`.

The Go workspace (`go.work`, 18 modules) builds and tests green; see [CLAUDE.md](#) for the canonical build/test commands. Standard `cmd/` + `internal/` layout. Roughly a dozen evidence-gated HRDs remain open in [docs/Issues.md](#) (live-integration credentials + later-iteration channels); the bulk of the catalogue migrated to [docs/Fixed.md](#).

Mission

Herald sits between systems that **emit events** and humans/services that **want to be told about them**. It guarantees:

- Every event is delivered to every configured channel for its category.
- Duplicate suppression and routing are explicit, not accidental.
- Delivery failures on one channel never block delivery to other channels.

Deployment model

Herald is designed to be consumed as a **git submodule** of a larger project (the "consuming project"). The consuming project provides:

- The **Helix Constitution** submodule at its own `<consuming-project>/constitution/`.
- Its own CI / build orchestration that drives Herald.
- Any project-wide configuration (credentials, deployment targets, channel registries).

Herald therefore **does not carry its own copy** of the constitution. See [docs/guides/CONSTITUTION_INHERITANCE.md](#) for the full rationale and the discovery contract that lets Herald locate the constitution at runtime from any nested depth.

Governance — Helix Constitution inheritance

Herald inherits unconditionally from the [Helix Universal Constitution](#). Inheritance is enforced by a paired gate + mutation meta-test under `tests/` (see below). Herald-specific extensions live in [docs/guides/HERALD_CONSTITUTION.md](#); there are currently no overrides of any universal clause.

Key invariants Herald inherits from the constitution:

- **No bluffing** — every PASS carries positive evidence (§11.4).
- **Mutation-paired gates** — every new gate has a paired mutation proving it catches regressions (§1.1).
- **No guessing language** — `likely`, `probably`, `maybe`, `seems`, `appears` are forbidden when reporting causes (§11.4.6).
- **Credentials never tracked** — `.env` git-ignored, runtime-load only (§11.4.10).
- **Multi-upstream push** — every commit fans out to GitHub + GitLab + GitFlic + GitVerse (§2.1).
- **Hardlinked backup before destructive ops** (§9).

The constitution lives at <https://github.com/HelixDevelopment/HelixConstitution> and is mirrored to GitLab, GitFlic, and GitVerse.

Repository layout

```
Herald/
├── README.md                # this file
├── CLAUDE.md                # guidance for Claude Code
├── AGENTS.md                # guidance for generic CLI
├── LICENSE
├── .gitignore
├── docs/
│   ├── guides/
│   │   ├── HERALD_CONSTITUTION.md    # Herald's project constitution
│   │   └── CONSTITUTION_INHERITANCE.md # operator/agent guide for
│   ├── specs/
│   │   └── mvp/
│   │       ├── specification.V4.md    # active spec (operator-prod
│   │       └── archive/
```

```

├── specification.V1.md      # historical, superseded
├── specification.V2.md      # historical, superseded
├── upstreams/
│   ├── GitHub.sh
│   ├── GitLab.sh
│   ├── GitFlic.sh
│   └── GitVerse.sh
├── tests/
│   ├── test_constitution_inheritance.sh      # inheritance gate
│   └── test_constitution_inheritance_meta.sh  # paired mutation meta-test

```

Quickstart for developers and agents

1. Place a copy of the Helix Constitution alongside Herald

If you cloned Herald standalone (not as a submodule of a larger project), put a clone of the constitution **next to** Herald (not inside it — see [§104 of the Herald Constitution](#)):

```
git clone git@github.com:HelixDevelopment/HelixConstitution.git \
    $(dirname "$PWD")/constitution
```

This is only needed for standalone work. When Herald is consumed as a submodule of a larger project, that project already provides <parent>/constitution/.

2. Verify the inheritance contract

```
bash tests/test_constitution_inheritance.sh      # gate (7
    invariants)
bash tests/test_constitution_inheritance_meta.sh  # paired §1.1
    mutation proof
```

Both MUST exit 0. The gate prints PASS ... / FAIL ... per invariant and a summary line. The meta-test prints ✓ META-TEST PASS when the gate correctly fails on a mutated constitution.

If I1 fails (constitution not found), follow the message — clone the constitution alongside Herald.

3. Read the inherited rules

In this order, read fully before submitting any change:

1. <discovered-constitution>/CLAUDE.md + Constitution.md — universal Helix rules.
2. <discovered-constitution>/AGENTS.md — anti-bluff, no-guessing, paired mutations.
3. This README — Herald overview.
4. [CLAUDE.md](#) / [AGENTS.md](#) — Herald-specific guidance.
5. [docs/guides/HERALD_CONSTITUTION.md](#) — Herald's articles §101–§106.
6. [docs/guides/CONSTITUTION_INHERITANCE.md](#) — the discovery contract and gate semantics.
7. [docs/specs/mvp/specification.V4.md](#) — current spec (active). Historical V1/V2 in [docs/specs/mvp/archive/](#).

Operator setup guides

Every supported messenger and every supported LLM / agent dispatcher has its own step-by-step setup guide under [docs/guides/](#). **Live** ones are detailed end-to-end (obtain credentials → set env vars → verify integration tests → troubleshooting). **Planned** ones are placeholder stubs that reserve the env-var names so your `.env` stays stable as features land.

Subscribers speak plain natural language — no command syntax. Whichever messenger they use, subscribers do NOT need to learn any command syntax (no `COMMAND:` prefix). They send a clear message in their own words and the System determines the intent via a three-tier discipline (command-recognition fast-path → Claude Code intent inference → a `clarify` reply-tag-and-ask fallback), never guessing and never ignoring a message. Authoritative contract: [docs/design/INTENT_RECOGNITION.md](#); operator-facing detail in [docs/guides/MESSENGER_CHANNELS.md](#) §6B.

Messengers

Channel	Status	Guide
Telegram	LIVE (HRD-011 code complete; awaiting live evidence)	docs/guides/messengers/TELEGRAM.md
Slack	Planned (V2)	docs/guides/messengers/SLACK.md
Email (SMTP + Resend)	Planned (V2)	docs/guides/messengers/EMAIL.md
Max	Planned (V2)	docs/guides/messengers/MAX.md
Microsoft Teams	Planned (V3)	docs/guides/messengers/TEAMS.md
Lark / Feishu	Planned (later)	docs/guides/messengers/LARK.md
Discord	Planned (later)	docs/guides/messengers/DISCORD.md
WhatsApp Business	Planned (later)	docs/guides/messengers/WHATSAPP.md
Viber	Planned (later)	docs/guides/messengers/VIBER.md

LLM / agent dispatchers

Dispatcher	Status	Guide
Claude Code	LIVE (HRD-012 Fixed — live E18 evidence captured)	docs/guides/dispatchers/CLAUDE_CODE.md
OpenCode	Planned	docs/guides/dispatchers/OPENCODE.md
Aider	Planned	docs/guides/dispatchers/AIDER.md
Gemini	Planned	docs/guides/dispatchers/GEMINI.md

Dispatcher	Status	Guide
Cursor	Planned	docs/guides/dispatchers/CURSOR.md
Anthropic Managed Agent	Planned	docs/guides/dispatchers/ANTHROPIC.md

Per-flavor operator guides

Each Herald flavor binary ships with a nano-detail operator reference under <docs/guides/>, documenting every subcommand the built binary surfaces (`<flavor> --help`), the env/credentials each needs, real example invocations, and which subcommands are live vs not-yet-implemented (all anti-bluff — derived from running the actual binary, nothing invented).

Flavor	Role	Guide
pherald	Project Herald — the richest flavor: <code>serve/listen/watch/migrate/wizard/commit-push</code> + the §43 GitOps commands	docs/guides/PHERALD.md
sherald	System Herald — <code>serve (/v1/safety_state)</code> + the host-safety guard commands (<code>destructive-guard</code> , <code>force-push-gate</code> , <code>mem-budget-watch</code> , <code>backup-snapshot</code> , <code>sysctl</code>)	docs/guides/SHERALD.md
cherald	Constitution Herald — <code>serve (/v1/compliance)</code> + the §11.4 compliance-check catalogue (<code>creds-scan</code> , <code>docs-sync</code> , <code>composite-gate</code> , <code>readme-sync</code> , ...)	docs/guides/CHERALD.md
bherald	Build Herald — CLI-only CI/test gates: <code>evidence-capture</code> (§11.4.2) + <code>test-tier-verify</code> (§40.2 8-tier matrix); <code>gate-retest</code> not-yet-implemented (HRD-045)	docs/guides/BHERALD.md
rherald	Release Herald — CLI-only release path: <code>changelog-generate</code> (§5), <code>tag-mirror</code> (§4 cross-mirror parity), <code>gate-retest</code> (§11.4.40 pre-tag retest)	docs/guides/RHERALD.md
iherald	Incident Herald — <code>serve</code> ; <code>POST /v1/webhooks/page</code> is a LIVE JWT-gated escalation handler (drives the bindings Pipeline → emits the escalation CloudEvent → 202 + Receipt; third-party pager egress is a follow-up subscriber)	docs/guides/IHERALD.md
scherald	Status-Check Herald — CLI-only <code>status-digest</code> (§11.4.45 rollup)	docs/guides/SCHERALD.md
qaherald	QA Herald — Herald's autonomous QA bot: <code>run</code> (scenario harness → docs/qa evidence), <code>mtproto</code> session lifecycle (<code>login/whoami/logout</code>), <code>lifecycle</code> (15-scenario driver, SKELETON)	docs/guides/QAHERALD.md

Credentials master guide

docs/guides/OPERATOR_CREDENTIALS.md is the umbrella reference covering:

- The 12-factor resolution order (CLI flag > shell exports > `.env` > defaults)
- How to set credentials via `~/ .bashrc` / `~/ .zshrc` (shell-export path) AND `.env` (project-local path) — both supported per spec V3 §3.3
- Pre-commit secrets audit checklist (`.env` not tracked; no obvious-format secrets; `.env.example` only has placeholders)
- Quickstart-compose vs native pherald operating modes

- The `HERALD_<CHANNEL>_OPERATOR_USERNAME` operator env var (e.g. `HERALD_TGRAM_OPERATOR_USERNAME=@milos85vasic`) that drives workable-item created_by/assigned_to attribution + notification @-tagging — authoritative contract [docs/design/PARTICIPANT_ATTRIBUTION.md](#), full behaviour in [docs/guides/WORKABLE_ITEMS_INTEGRATION.md](#) §3.6–§3.8
- Troubleshooting (SQLSTATE 28P01, "DSN not set", SKIP-with-reason logic per §11.4.3, ...)

Per Universal Constitution §11.4.10: `.env` files **MUST NEVER be committed**. The repo's `.gitignore` already covers `.env`. The committed `quickstart/.env.example` is the only credentials-file that lives in git, and it contains only placeholder values.

Active blockers — operator action required

[docs/requirements/blockers/missing_env_variables.md](#) — the step-by-step MTPROTO credential setup the operator must complete to unblock Wave 8 Track B (the full-automation rewrite of `TestSubscribe_LiveBotAPI`, `tests/test_wave6_live_loop.sh`, and Wave 6.5 lifecycle scenarios per HelixConstitution §11.4.98 + Herald §108.m). Available in Markdown / HTML / PDF / DOCX. **Required for §11.4.98 compliance — release-gate item.**

[docs/audits/full-automation-114-98-audit-2026-05-28.md](#) — the canonical classification of every Herald test against §11.4.98 (794 tests audited: 758 COMPLIANT-hermetic, 32 COMPLIANT-with-creds-bootstrap, 4 NON-COMPLIANT-manual-dep, 1 STRUCTURALLY-BROKEN). NON-COMPLIANT items have until 2026-06-27 (T+30 days) to be rewritten before graduating to §11.4.90 Obsolete.

Mirror & push convention

Herald is mirrored to four hosts. The `origin` remote is **fan-out**: one fetch URL + four push URLs. A single `git push origin main` propagates to every mirror in one operation.

Remote name URL

<code>github</code>	<code>git@github.com:vasic-digital/Herald.git</code>
<code>gitlab</code>	<code>git@gitlab.com:vasic-digital/herald.git</code>
<code>gitflic</code>	<code>git@gitflic.ru:vasic-digital/herald.git</code>
<code>gitverse</code>	<code>git@gitverse.ru:vasic-digital/Herald.git</code>
<code>origin</code>	(fetch from <code>github</code> ; push fans out to all four)

Each entry under `upstreams/` is a shell script that exports a single `UPSTREAMABLE_REPOSITORY=...` URL and is meant to be **sourced**, not executed for its output. Capitalization matches each host's brand (GitFlic, GitVerse); do not normalize to lowercase or collapse into one file — external mirror-push tooling keys on the per-file split.

If you ever need to rebuild the fan-out configuration, the constitution submodule ships `install_upstreams.sh` that consumes `Upstreams/*.sh` declarations and configures git remotes accordingly; the same pattern can be adapted for Herald.

License

[LICENSE](#) — see file for terms.

Contact / contribution

Substantive contributions land via PRs on GitHub; mirrors are read-only for external consumers. Inheritance rules and the gate apply to every PR.

Sources verified

Per HelixConstitution §11.4.99 + Herald §108.n (Latest-Source Documentation Cross-Reference Mandate). This README is a gateway document — the substantive operator-facing instructions live in the per-channel / per-dispatcher guides linked above. Each linked guide carries its own `## Sources verified` footer covering the external services it documents. The cross-references below cover the external claims this README makes directly.

Last verified: 2026-05-28

Source	URL / path	Authored / verified
HelixConstitution	https://github.com/HelixDevelopment/HelixConstitution	§"Governance" (constitution authority + canonical URL); §"Quickstart" step 1 (clone-alongside command); §"Inherited invariants" list (§11.4 anti-bluff, §1.1 paired mutations, §11.4.6 no-guessing language, §11.4.10 credentials-never-tracked, §2.1 multi-upstream push, §9 hardlinked-backup). §"Messengers" table — Telegram channel status (LIVE — HRD-011 + HRD-100). The substantive setup steps are in docs/guides/messengers/TELEGRAM.md + docs/guides/TELEGRAM.md which carry their own §11.4.99 footers. §"LLM / agent dispatchers" table — Claude Code status (LIVE — HRD-012 Fixed). The substantive setup steps are in docs/guides/dispatchers/CLAUDE_CODE.md which carries its own §11.4.99 footer. §"Active blockers" link target — that document carries the canonical Telegram-userbot safety walkthrough (the <code>recover@telegram.org</code> pre-login email; no VoIP / Google Voice / Twilio / TextNow numbers; one phone = one <code>api_id</code> forever; Short-name STRICTLY alphanumeric — underscores REJECTED; <code>ratelimit</code> + <code>floodwait</code> middlewares) with its own §11.4.99 footer. §"Status" claims about Herald implementation maturity; §"Mission" canonical statement; §"Repository
Telegram official Bot API documentation	https://core.telegram.org/bots/api	
Anthropic — Claude Code documentation	https://docs.anthropic.com/claude-code	
Herald active blockers — MTProto credential setup	docs/requirements/blockers/missing_env_variables.md	
Herald spec V3 (source of truth)	docs/specs/mvp/specification.V4.md	

Source

URL / path

Authored / verified

layout"; the V1→V2→V3 supersession chain. The spec itself carries its own §11.4.99 footer covering the external service contracts its design decisions depend on.

Re-verification cadence (per §11.4.99 (C)): This README does not contain operator-actionable external-service instructions directly — those are in the linked guides, each with its own cadence. README-level re-verification is **on Herald structural changes** (repo-layout changes, new flavors, new mirror hosts, new linked guides) — no time-bound staleness. Linked guides **MUST** be kept current per their own footers.